

编译原理大作业第二部分（语法分析）报告

学号：524031910374

姓名：曾清

本次实验在 Lab 1 词法分析的基础上，进一步实现了 Pony 编译器框架中的语法分析器（Parser）。语法分析的核心任务是接收 Lexer 产生的 Token 流，采用递归下降（Recursive Descent）的分析方法，将其转化为结构化的抽象语法树（AST）。

在本次实验中，我重点完成了 `parseDeclaration`、`parseIdentifierExpr` 和 `parseBinOpRHS` 三个核心函数的构建，分别实现了对变量声明（包含张量形状类型）、标识符与函数调用、以及带优先级的二元操作符表达式的解析。

本报告共分为五个部分：

1. `parseDeclaration()` 的实现思路与方法
2. `parseIdentifierExpr()` 的实现思路与方法
3. `parseBinOpRHS()` 的实现思路与方法
4. 实验结果展示与分析
5. 遇到的问题与解决

一、`parseDeclaration()` 的实现

功能描述

该函数负责解析变量的声明语句。根据 Pony 语言的设定，变量声明以 `var` 关键字开头，并且支持以下三种初始化形式：

1. `var a = [[1, 2, 3], [4, 5, 6]];`（无显式形状声明）
2. `var a<2,3> = [1, 2, 3, 4, 5, 6];`（变量名后接形状声明）
3. `var <2,3> a = [1, 2, 3, 4, 5, 6];`（变量名前接形状声明）

实现思路

处理这一逻辑的核心难点在于，吃掉 `var` 关键字后，后续 Token 的顺序会因声明方式的不同而产生分叉。因此，我采用了基于前瞻 Token（Lookahead）的 `if-else` 显式分支设计，以消除逻辑交错冲突：

第一步：验证 `var` 关键字。 检查当前 Token 是否为 `tok_var`，若是则消耗掉。

第二步：基于下一个 Token 进行分支预测。

- **分支 A（类型 3）：** 如果紧接着的 Token 是 `<`，说明用户采用了第三种声明方式。此时必须先调用 `parseType()` 解析出变量的形状存入 `type`，随后再断言并读取标识符作为变量名。
- **分支 B（类型 1 或 2）：** 如果紧接着的 Token 是 `tok_identifier`，则先将其读取为变量名。随后再往后看一眼：如果跟着 `<`，则说明是第二种声明，调用 `parseType()` 解析形状；如果没有 `<`，则是第一种声明，保持 `type` 为空。

第三步：收尾与 AST 节点构建。 针对第一种未显式提供形状的情况，兜底分配一个默认的空 `VarType`。最后验证并吃掉等号 `=`，调用 `parseExpression()` 解析右侧的初始化表达式，并返回封装好的 `VarDeclExprAST` 节点。代码结构如下：

```
if (lexer.getCurToken() == '<') {
    // 处理类型 (3): var <2,3> a = ...
    type = parseType();
    if (!type) return nullptr;
    if (lexer.getCurToken() != tok_identifier)
        return parseError<VarDeclExprAST>("identifier", "in variable declaration");
    id = lexer.getId().str();
    lexer.consume(tok_identifier);
} else if (lexer.getCurToken() == tok_identifier) {
    // 处理类型 (1)(2): var a = ... 或 var a<2,3> = ...
    id = lexer.getId().str();
    lexer.consume(tok_identifier);
    if (lexer.getCurToken() == '<') {
        type = parseType();
        if (!type) return nullptr;
    }
}
```

二、`parseIdentifierExpr()` 的实现

功能描述

该函数负责解析以标识符 (Identifier) 开头的表达式。在 Pony 语言中，这类表达式可能是简单的变量引用（如 `a`），也可能是普通函数调用（如 `foo(a, b)`），或是内置的打印函数调用（如 `print(a)`）。

实现思路

解析过程本质上是一个循序渐进的结构试探：

第一步：提取标识符。 先通过 `lexer.getId().str()` 获取变量名或函数名，并吃掉当前的 `tok_identifier`。

第二步：区分变量与函数调用。 检查下一个 Token，如果不是左括号 `(`，说明它仅仅是一个普通的变量引用，直接返回 `VariableExprAST` 即可结束解析。

第三步：解析函数参数列表。 如果遇到了 `(`，说明是函数调用。进入一个 `while` 循环，只要当前 Token 不是 `)`，就不断调用 `parseExpression()` 解析参数，并将其推入 `args` 数组中。如果参数之间有逗号 `,`，则将其吃掉以进行分隔。循环结束后吃掉右括号 `)`。

第四步：特判 `print` 函数。 判断解析到的标识符名字是否为 `"print"`。若是，则进行特殊的参数数量安全性检查：`print` 必须有且仅有一个参数，即 `args.size() != 1` 时报错拦截。若合法，返回 `PrintExprAST`；对于其他普通函数调用，则返回通用的 `CallExprAST`。

三、parseBinOpRHS() 的实现

功能描述

此函数是算符优先分析法（Operator Precedence Parsing）的核心，用于处理二元表达式的右半部分。它通过优先级门槛（`exprPrec`）解决诸如 `a + b * c` 这样的算符结合优先级问题，并在本次实验中额外扩展了对矩阵乘法算符 `@` 的支持（优先级同 `*`）。

实现思路

为了确保高优先级的算符能更紧密地结合，函数使用了一个带门槛检测的 `while` 循环递归架构：

第一步：门槛过滤。 获取当前 Token 的运算符优先级 `tokPrec`。如果其优先级低于外层传入的基准优先级 `exprPrec`（或者它根本不是一个运算符），说明当前的二元表达式在这一层已经到头，直接将左侧已构建的 `lhs` 返回给外层。

第二步：提取右侧初级表达式。 越过门槛后，吃掉当前运算符，并调用 `parsePrimary()` 解析出操作符右侧紧挨着的原子表达式（如数字或变量），记为 `rhs`。

第三步：前瞻与递归结合（Lookahead）。 这一步是算法的灵魂。偷看下一个待处理算符的优先级 `nextPrec`。如果 `tokPrec < nextPrec`（例如当前是 `+`，下一个是 `*`），说明右边的算符结合得更紧密。此时需要**提升门槛**，通过传递 `tokPrec + 1` 递归调用 `parseBinOpRHS`，强制先将右侧高优先级的部分计算并缝合成一个整体的 `rhs`。

第四步：构建二元节点。 递归回溯后，将当前的运算符与 `lhs`、`rhs` 组装成一个新的 `BinaryExprAST` 节点。这个新节点在下次循环迭代中将作为更庞大的左半部分继续参与运算。

四、实验结果展示与分析

为了验证语法分析器的正确性，在 `emit=ast` 模式下，分别运行 `test_8.pony ~ test_12.pony` 都得到了期望的预期结果；在 `emit=jit` 模式下，运行 `test_11.pony` 也得到了正确的输出。在报告中，我选取了 `test_8.pony(emit=ast)`、`test_9.pony(emit=ast)` 和 `test_11.pony(emit=jit)` 三个测例为代表进行实验结果的展示与分析，这三个例子分别验证了 AST 树的正确构建、语法错误的精准拦截以及后续代码生成与 JIT 引擎执行的正确性。

1. test_8.pony（验证 AST 生成正确性）

测试命令： `../build/bin/pony ../test/test_8.pony -emit=ast`

源码：

```
# ../build/bin/pony ../test/test_8.pony -emit=ast
# ../build/bin/pony ../test/test_8.pony -emit=jit

def main() {

    var a = [[1, 2, 3], [4, 5, 6]];
    var <2,3> b = [1, 2, 3, 4, 5, 6];
    print(a);
    print(b);
}
```

```
}
```

输出:

```
Module:
  Function
    Proto 'main' @../..test/test_8.pony:4:1
    Params: []
    Block {
      VarDecl a<> @../..test/test_8.pony:6:3
        Literal: <2, 3>[ <3>[ 1.000000e+00, 2.000000e+00, 3.000000e+00], <3>[
4.000000e+00, 5.000000e+00, 6.000000e+00]] @../..test/test_8.pony:6:11
      VarDecl b<2, 3> @../..test/test_8.pony:7:3
        Literal: <6>[ 1.000000e+00, 2.000000e+00, 3.000000e+00, 4.000000e+00,
5.000000e+00, 6.000000e+00] @../..test/test_8.pony:7:17
      Print [ @../..test/test_8.pony:8:3
        var: a @../..test/test_8.pony:8:9
      ]
      Print [ @../..test/test_8.pony:9:3
        var: b @../..test/test_8.pony:9:9
      ]
    } // Block
```

分析: 从 AST 树结构可以看出, 语法分析器完美识别了两种不同的变量声明方式: `a` 被识别为未显式指定形状的声明, 即 `a<>`, 而 `<2,3> b` 成功解析出了 `b<2, 3>` 的 `VarType`。字面量数组也被正确展开, `print` 语句被准确转化为带有单一子变量的 AST 节点。结果完全符合预期。

2. test_9.pony (验证错误报错机制)

测试命令: `../build/bin/pony ../test/test_9.pony -emit=ast`

源码:

```
# ../build/bin/pony ../test/test_9.pony -emit=ast

def main() {

  var [2, 3] b = [1, 2, 3, 4, 5, 6];
  print(b);

}
```

输出:

```
Parse error (5, 7): expected 'identifier' in variable declaration but has Token 91
 '['
```

```
Parse error (5, 7): expected 'nothing' at end of module but has Token 91 '['
```

分析：源代码中使用了错误的形状声明格式 `[2, 3]`（按照语法定义应当是 `<2, 3>`）。分析器在处理 `var` 后，既没有遇到 `<`，遇到的也不是合法的标识符，于是立刻触发了 `parseDeclaration` 中的防御分支，准确打印出了期望 `identifier` 却遇到了 `[` 的错误信息，并在最外层阻止了 AST 的构建，这证明了语法分析器具有良好的鲁棒性。

3. `test_11.pony`（验证 JIT 综合执行结果）

测试命令： `../build/bin/pony ../test/test_11.pony -emit=jit`

源码：

```
# ../build/bin/pony ../test/test_11.pony -emit=ast
# ../build/bin/pony ../test/test_11.pony -emit=jit

def main() {

    var a = [[1, 2, 3], [4, 5, 6]];
    var b<2, 3> = [1, 2, 3, 4, 5, 6];
    var c = [[1, 2, 3], [4, 5, 6]];
    var d<2, 3> = [1, 2, 3, 4, 5, 6];
    var e = (a+c)*(b+d);
    print(e);

}
```

输出：

```
4.000000 16.000000 36.000000
64.000000 100.000000 144.000000
```

分析：该测例包含了复杂的变量多态声明、二元运算的嵌套（括号干预优先级）、点乘运算，并最终通过 `print` 输出计算结果。通过 JIT 执行，它不仅证明了 AST 树的形态绝对正确（否则 MLIR Dialect 无法生成），也验证了我实现的 `parseBinOpRHS` 对优先级和算符的嵌套解析是绝对精准的，最终得到了正确的矩阵乘加数学结果。

五、遇到的问题与解决

在实现 `parseDeclaration` 和 `parseIdentifierExpr` 时，我遇到过一个难以绕过的 C++ 编译报错问题。当时我的代码如下：

```
std::string id;
id = lexer.getId();
```

使用 `cmake --build .` 编译时，终端弹出了一大段包含 `<string>` 模板展开的报错。最核心的错误提示是：
`no known conversion for argument 1 from 'llvm::StringRef' to 'const std::string&'`

排查与解决： 经过查找和学习，我发现这是由于 LLVM 框架与 C++ 标准库底层数据结构的设计差异引起的。Pony 的词法分析器 `lexer.getId()` 返回的类型是 `llvm::StringRef`。在 LLVM 生态中，`StringRef` 非常轻量，仅仅包含一个指针和一个长度，它不占有内存的所有权。而 `std::string` 是真正的堆内存字符串对象。为了性能和内存安全考虑，C++ 编译器严禁将 `StringRef` 隐式赋值或转化给 `std::string`，以防止造成悬空指针（Dangling Pointer）或不明确的内存拷贝操作。

解决该问题的方法是显式地触发内存拷贝与转换。我在调用返回值之后增加了一个 `.str()` 方法：

```
id = lexer.getId().str();
```

这一操作会告诉 `StringRef` 将其内部指向的字符数组数据显式地拷贝并实例化为一个新的 `std::string` 对象。修改后，问题圆满解决，代码顺利通过了编译。